Q1:

Complex Scenario-Based Problem on Learning Algorithm, Loss Function, Chain Rule, and Gradient Descent

A company is developing a deep learning model to predict the selling price of used cars.

A single neuron is used for prediction with the following parameters:

* **Input feature (car age): $x = 4$**
* **Actual selling price: $y = 15$**
* **Initial weight: $W = 2$**
* **Bias: $b = 1$**
* **Learning rate: $\eta = 0.1$**
* **Activation function: Identity, $f(z)=z$**
* **Loss function: Mean Squared Error (MSE)**

The model is defined as:

$$\hat{y} = Wx + b$$

and the loss function is:

$$L=\frac{1}{2}(y-\hat{y})^2$$

Questions

1. Perform forward propagation and calculate the predicted output (\hat{y}).

2. Compute the loss using the MSE formula.

3. Using the chain rule, derive the gradient \frac{\partial L}{\partial W}.

4. Calculate the updated weight after one iteration of gradient descent.

5. Calculate the updated bias.

6. If the learning rate is increased from 0.1 to 1.0, discuss how it affects convergence and model stability.

7. If the activation function is changed from the identity function to ReLU, explain whether the prediction and gradient computation will change for this input.

8. Suggest two methods to reduce the loss further without changing the dataset.

9. Explain how backpropagation uses the chain rule to update parameters in a multilayer neural network.

10. If another feature (car mileage) is added, explain how the learning algorithm and weight updates will change.

CODE:

```
x = 4

y = 15

W = 2

b = 1

learning_rate = 0.1


y_hat = W * x + b

loss = 0.5 * (y - y_hat) ** 2


dL_dW = (y_hat - y) * x

dL_db = (y_hat - y)
```

```
new_W = W - learning_rate * dL_dW
new_b = b - learning_rate * dL_db

print("1. Predicted Output (y_hat):", y_hat)
print("2. Loss:", loss)
print("3. Gradient dL/dW:", dL_dW)
print("4. Updated Weight:", new_W)
print("5. Updated Bias:", new_b)

print("\n6. Effect of Increasing Learning Rate to 1.0")
print("Faster learning, but may overshoot the minimum, oscillate, or diverge, making training unstable.")

print("\n7. Identity Function vs ReLU")
if y_hat > 0:
    print("Since z =", y_hat, "> 0, ReLU output is the same as the identity function.")
    print("Prediction and gradient remain unchanged.")
else:
    print("ReLU output becomes 0 and the gradient becomes 0.")

print("\n8. Two Methods to Reduce Loss")
print("1. Train for more epochs.")
print("2. Use a better optimizer or tune the learning rate.")

print("\n9. Backpropagation")
print("Backpropagation uses the chain rule to compute gradients of the loss with respect to each parameter and updates the weights and bias using gradient descent.")
```

```
print("\n10. Adding Another Feature (Car Mileage)")
print("Prediction becomes: y_hat = W1 * Age + W2 * Mileage + b")
print("Each weight has its own gradient and is updated separately during gradient descent.")
```

OUTPUT:

1. Predicted Output (y_hat): 9

2. Loss: 18.0

3. Gradient dL/dW: -24

4. Updated Weight: 4.4

5. Updated Bias: 1.6

6. Effect of Increasing Learning Rate to 1.0

Faster learning, but may overshoot the minimum, oscillate, or diverge, making training unstable.

7. Identity Function vs ReLU

Since z = 9 > 0, ReLU output is the same as the identity function.

Prediction and gradient remain unchanged.

8. Two Methods to Reduce Loss

1. Train for more epochs.

2. Use a better optimizer or tune the learning rate.

9. Backpropagation

Backpropagation uses the chain rule to compute gradients of the loss with respect to each parameter and updates the weights and bias using gradient descent.

10. Adding Another Feature (Car Mileage)

Prediction becomes: y_hat = W1 * Age + W2 * Mileage + b

Each weight has its ow**n** gradient and is updated separately during gradient descent.